



UD1

ARQUITECTURA DE APLICACIONES WEB

1.3 Tecnologías y lenguajes de programación entorno servidor

MP_0613

Desarrollo web en entorno
servidor

Aplicaciones estáticas y dinámicas

Una de las diferencias fundamentales en el desarrollo web es la existente entre **contenido**

estático y contenido dinámico

- **Contenido estático:** Un contenido estático es aquel que el servidor puede entregar directamente sin tener que generarlo específicamente para cada petición. Por ejemplo El servidor puede almacenar este archivo y enviarlo al navegador cuando se solicite.

```
HTML
<!DOCTYPE html>
<html>
<body>
<h1>Mi página web</h1>
<p>Bienvnids a mi sitio web</p>
</body>
</html>
```

- **Contenido dinámico:** El contenido dinámico se genera o modifica en función de diferentes datos o circunstancias.

Por ejemplo, imaginemos una página que muestra:

Bienvenida, Sandra

Si otro usuario inicia sesión podría mostrar

Bienvenido, Juan

La página no tiene necesariamente un contenido idéntico para todos los usuarios.

Otros ejemplos de contenido dinámico son:

- Fecha y hora.
- Nombre del usuario.
- Resultados de una búsqueda.
- Productos disponibles.
- Noticias.
- Información almacenada en una base de datos.
- Resultados de un formulario.

Para generar este contenido podemos utilizar programación en el servidor.

Ventajas de la generación dinámica de páginas

Frente al contenido estático, generar el contenido de forma dinámica en el servidor aporta ventajas importantes:

- **Personalización:** se puede adaptar la respuesta a cada usuario, sesión o petición concreta.
- **Actualidad de los datos:** el contenido se genera a partir de la información disponible en el momento de la petición (bases de datos, hora del sistema, resultados de búsqueda), sin necesidad de reescribir páginas manualmente.
- **Mantenimiento reducido:** basta con actualizar los datos de origen (por ejemplo, una tabla de productos) para que el cambio se refleje en todas las páginas que lo muestran, sin tocar el código HTML de cada una.
- **Interactividad:** permite procesar formularios, gestionar usuarios y responder de forma distinta según las acciones que realice cada visitante.

Como contrapartida, el contenido dinámico exige más recursos del servidor (tiempo de procesamiento, acceso a bases de datos) que el contenido estático, que el servidor puede entregar directamente sin ningún procesamiento adicional.

Procesamiento en cliente y procesamiento en servidor

En una aplicación web podemos ejecutar código en diferentes lugares.

Esta distinción es especialmente importante en Desarrollo Web en Entorno Servidor.

- **Procesamiento en el cliente:** El código se ejecuta en el navegador del usuario, el principal lenguaje utilizado en este caso es **JavaScript**. Por ejemplo, podemos utilizar JavaScript para verificar que un formulario contiene todos los campos necesarios antes de su envío.

- **Procesamiento en el servidor:** El código se ejecuta en el servidor. Algunas de las tecnologías utilizadas para ellos son: **PHP, JSP, Java, Node.js** y **ASP.NET**. Una característica fundamental es que **el navegador recibe el resultado del procesamiento, no el código del servidor**.

Por ejemplo, si en el servidor tenemos:

```
PHP
<?php
echo "Hola mundo";
?>
```

El navegador recibirá:

```
HTML
Hola mundo
```

Programación en el servidor

Para crear aplicaciones que se ejecuten en una capa intermedia podemos utilizar diferentes lenguajes de programación. Pero, además de un lenguaje, se necesita algún tipo de tecnología o librería especial de apoyo.

Con esta tecnología, podremos resolver los diferentes problemas que nos encontramos al crear este tipo de programas, como la gestión de los datos de usuario, el mantenimiento de la información, el manejo de peticiones concurrentes, etc.

Existen muchos lenguajes y tecnologías para crear aplicaciones del lado del servidor y resolver toda la problemática que este tipo de desarrollo conlleva.

Entre estas tecnologías y lenguajes destacaremos los siguientes:

- Java/Java EE.
- PHP.
- Node.js
- ASP/ASP.NET.
- Ruby on Rails.

Estos lenguajes se emplean para la creación de la aplicación que se ejecutará en el servidor de la capa intermedia. Al cliente le resulta indiferente cuál de estas tecnologías se emplee, pues, independientemente de la que se utilice, lo que se va a generar y enviar en la respuesta HTTP serán cadenas de texto en formato HTML interpretables por cualquier navegador.

Java / Java EE

La combinación del lenguaje Java con la especificación Java EE constituye un estándar de facto en la creación de aplicaciones web.

Sin duda alguna, Java EE es la **tecnología más empleada** por las empresas de desarrollo software para la creación de aplicaciones web, tanto en el entorno internet como en el intranet.

Java EE es una especificación de Oracle que se compone de un conjunto de tecnologías y librerías para la creación de aplicaciones web en Java, entre otras cosas, proporciona una serie de componentes con los que construir nuestra aplicación. Estos componentes son:

- Servlets
- Java Server Pages (JSP). Enterprise
- JavaBeans (EJBs).

Cada uno de estos componentes desempeña un determinado rol y es adecuado para una determinada tarea, de modo que combinándolos podemos crear aplicaciones robustas y escalables.

Más adelante, analizaremos su construcción y funcionamiento.

Desde la aparición de los primeros programas informáticos, tanto el desarrollo software como los paradigmas y modelos de programación han evolucionado enormemente. Las arquitecturas de desarrollo, o maneras en las que se organizan los diferentes actores en una aplicación, también han tenido una gran evolución a lo largo de estos años.

Además de los componentes básicos, existen numerosos frameworks (o conjuntos de utilidades desarrolladas por empresas o comunidades de desarrolladores) orientados a simplificar y optimizar la construcción de aplicaciones basadas en Java EE. Entre estos frameworks tenemos **Spring**, **Java Server Faces (JSF)**, **Hibernate**, etc.

Al tratarse de un lenguaje multiplataforma, las aplicaciones Java/Java EE pueden desplegarse en servidores que cuenten con cualquier sistema operativo moderno, como Windows, Linux o

Mac. Eso sí, una aplicación Java EE requiere de la existencia de un software conocido como servidor de aplicaciones para poder ejecutarse.

PHP

PHP es un lenguaje de programación nacido expresamente para la **creación** de aplicaciones web del lado del servidor.

En una aplicación web creada con PHP, el código es insertado dentro de la propia página web, de modo que, cuando el cliente solicita esta página (al enviar una petición HTTP), dicho código se ejecuta. Como resultado, se envía una página totalmente formateada en HTML.

PHP proporciona un amplio abanico de instrucciones para acceder a bases de datos MySQL, lo que permite **generar dinámicamente páginas HTML a partir del contenido almacenado en estas bases de datos**, sin requerir de ninguna librería adicional, solo con instrucciones del propio lenguaje.

Una ventaja de PHP, respecto a otras tecnologías, es que el código PHP es **interpretado directamente por el servidor web**, es decir, no necesitamos instalar ningún programa especial en la capa intermedia (aparte del servidor web) para poder ejecutar una aplicación PHP.

Un ejemplo muy sencillo sería:

```
PHP
<?php
echo "Hola mundo";
?>
```

El código PHP se ejecuta en el servidor y el navegador recibirá el resultado:

```
HTML
Hola mundo
```

Otro ejemplo puede ser:

```
HTML
<body>
  <?php
    print "<h1>Mensajes</h1>\n";
    For ($i=1; $i<4; $i++){
      Print "<h2>Hola $i</h2>\n";
    }
    print "<p>Final</p>\n";
  ?>
</body>
```

El navegador recibirá:

```
HTML
<body>
  <h1>Mensajes</h1>
    <h2>Hola 1 </h2>
    <h2>Hola 2 </h2>
    <h2>Hola 3 </h2>
    <p>Final</p>
</body>
```

ASP / ASP.NET

La tecnología ASP fue **una de las primeras** en utilizarse para la creación de aplicaciones web del lado del servidor.

Desarrollada por **Microsoft en los años 90**, las *active server pages* se basan en la introducción de scripts del lado del servidor dentro de una página web, scripts que serán ejecutados en dicho servidor cuando la página sea solicitada desde el cliente, generando una cadena final de texto formateado en HTML. Por ejemplo:

```
ASP
<body>
  <h1>Mensajes</h1>
  <%for i=1 to 3%>
    <h2>Hola <%=i %></h2>
  <%next%>
  <p>Final</p>
</body>
```

El navegador recibirá:

```
HTML
<body>
  <h1>Mensajes</h1>
  <h2>Hola 1 </h2>
  <h2>Hola 2 </h2>
  <h2>Hola 3 </h2>
  <p>Final</p>
</body>
```

Se trata del mismo principio que hemos visto en PHP. De hecho, fueron las páginas ASP las primeras en introducir el concepto de scripts o bloques de código del lado del servidor.

En el caso de APS, el código de los scripts del servidor está escrito en lenguaje VBScript, que es una variante simplificada del clásico Visual Basic. A diferencia de PHP que, hoy en día, es un lenguaje totalmente orientado a objetos, VBScript es mucho más limitado y rudimentario, motivo por el cual, a principios del año 2000, Microsoft lanzó la tecnología **ASP.NET**.

ASP.NET forma parte de la plataforma .NET y, fundamentalmente, introduce dos grandes mejoras respecto a ASP:

- **Separación del código de la presentación:** En lugar de mezclar en una misma página los bloques de HTML con el código de servidor, ASP.NET permite separar este código en archivos independientes, lo que hace que las aplicaciones sean más fáciles de mantener y más escalables.
- **Utilización de lenguajes orientados a objetos:** En lugar de emplear el limitado VBScript, en la creación de aplicaciones ASP.NET puede emplearse cualquiera de los lenguajes orientados a objetos incluidos en la plataforma .NET, como C# o VB.NET.

Ruby on Rails

Ruby es un lenguaje de programación de alto nivel orientado a objetos, que posee gran cantidad de características para el proceso de textos y la generación de tareas.

Aunque fue lanzado a mediados de los 90, se trata de un lenguaje con gran potencial de futuro, gracias a la combinación de potencia y sencillez.

- **Ruby** ha adquirido gran popularidad desde que se utilizó en combinación con la plataforma Rails para la construcción de aplicaciones web.
- **Framework Rails** simplifica enormemente el desarrollo de aplicaciones web, ya que permite hacer supuestos sobre lo que cada desarrollador necesita para comenzar.

La unión de la sencillez de Rails y la potencia de Ruby hace de ambos una opción muy interesante para construir aplicaciones web del lado del servidor.

Node.js

A diferencia de PHP, JSP o ASP, Node.js no es un lenguaje embebido en HTML, sino un **entorno de ejecución de JavaScript en el servidor**, construido sobre el motor V8 de Google Chrome.

Su característica principal es el modelo **asíncrono y no bloqueante**: un mismo proceso puede atender muchas peticiones simultáneas sin crear un hilo por cada una, lo que lo hace especialmente eficiente en aplicaciones con muchas conexiones concurrentes (chats, APIs, aplicaciones en tiempo real). Al usar JavaScript también en el servidor, permite compartir el mismo lenguaje entre cliente y servidor, y cuenta con un ecosistema de librerías muy extenso a través de **npm**. El framework más utilizado sobre Node.js para crear aplicaciones web y servicios es **Express**.

Criterios de selección de tecnologías

Ninguna tecnología de programación en servidor es mejor que las demás en términos absolutos: la elección depende de los requisitos concretos del proyecto. Algunos criterios a valorar son la curva de aprendizaje, el rendimiento esperado, la escalabilidad, el tamaño y actividad de la comunidad, el coste (licencias, hosting) y el ecosistema de librerías disponible.

Tecnología	Curva de aprendizaje	Concurrencia / rendimiento	Comunidad / ecosistema	Cuándo suele elegirse
PHP	Baja	Media (mejora con FPM/OPcache)	Muy amplia, especialmente en hosting compartido	Proyectos pequeños/medianos, gestores de contenido, prototipos rápidos
Java / Java EE (Spring)	Alta	Alto (multihilo, tipado fuerte)	Muy amplia en entornos empresariales	Aplicaciones empresariales grandes, sistemas críticos, integraciones complejas
Node.js	Media	Muy alto en E/S concurrente (asíncrono)	Enorme (npm), muy activa	APIs REST, aplicaciones en tiempo real, microservicios
ASP.NET	Media-alta	Alto	Sólida en entornos Microsoft	Organizaciones ya integradas en el ecosistema .NET/Windows

A lo largo del módulo utilizaremos precisamente varias de estas familias de tecnologías para comprobar en la práctica estas diferencias: comenzaremos con PHP y JSP, continuaremos con Java y el framework Spring Boot y terminaremos con Node.js.